Tecnológico de Monterrey

16. Ingesta de datos

Ciencia de Datos II

Jorge Juvenal Campos Ferreira

juvenal.campos@tec.mx

Fuentes de los datos

Los datos que utilizamos pueden provenir de varias fuentes:

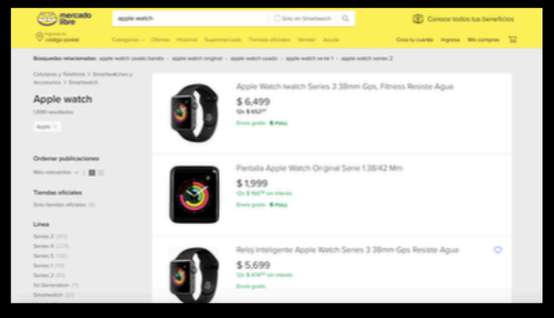
Datos Estructurados y no estructurados

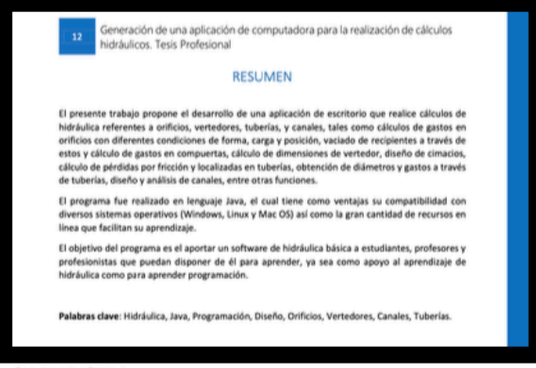
Los datos estructurados son datos presentados en una forma predefinida. Típicamente (pero no exclusivamente) se presenta de forma tabular, en el que las columnas son variables y las filas observaciones.

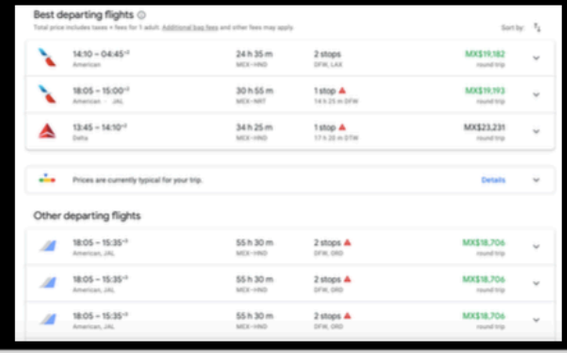
Entre estos datos tenemos los archivos de texto plano (*.csv, *.txt), los archivos de Office (*.docx, *.xlsx), los archivos que almacenan información geográfica (*.shp, *.geojson), entre otros, que tienen una estructura definida y son leídos por software de manera nativa.

Los datos no-estructurados. Son datos que **no tienen un modelo predefinido de datos o bien no está presentado de forma ordenada.** Típicamente son conjuntos de texto, imágenes, números, fechas que no están organizados de forma sistemática.

Entre estos se encuentran los *.pdfs (algunos), los *,pdfs fotografías, así como la información disponible en páginas de internet (catálogos, textos, etc).







Datos estructurados

Los datos estructurados.
Empaquetados y listos para utilizarse.

Datos no estructurados

Los datos no-estructurados.
La información esta ahí, pero hay que extraerla y empaquetarla.

1. Datos Abiertos

Datos Abiertos

"Filosofía y práctica que persigue que **determinados tipos de datos estén disponibles de forma libre para todo el mundo,** sin restricciones de derechos de autor, de patentes o de otros mecanismos de control".

Open Knowledge foundation

Datos Abiertos

[Los datos] son abiertos si satisfacen las condiciones siguientes:

1. **Disponibles y de fácil acceso.** (Disponible como un todo desde el principio, a un costo razonable de reproducción, preferentemente desde internet).
2. **Reutilizables y redistribuibles** (los datos deben ser provistos bajo términos que permitan reutilizarlos, redistribuirlos e integrarlos con otros conjuntos de datos).
3. **Facilitan la participación universal** (todos deben poder utilizar, reutilizar y redistribuir la información, sin discriminación en términos de esfuerzo, personas o grupos).

Datos Abiertos (léanlo después)

-Datos Abiertos

[Los datos] son abiertos si su forma de distribución satisface las condiciones siguientes:

1.. [Los datos] deben estar disponibles integralmente y a un coste de reproducción razonable, preferiblemente descargable de internet de manera gratuita. Los datos igualmente deben estar disponibles en una forma conveniente para ser modificables.
2.. La licencia no debe restringir a nadie la posibilidad de vender o distribuir los datos en sí mismos o formando parte de un paquete hecho de otros conjuntos de datos. Igualmente, la licencia no debe exigir un pago u otro tipo de cuota para esta venta o distribución.
3.. La licencia debe permitir hacer modificaciones y obras derivadas y debe permitir que estas sean distribuidas en las mismas condiciones que la obra original. La licencia puede imponer algún tipo de requerimiento referente al reconocimiento y a la integridad.
4. Se deben proporcionar los datos de tal manera que no haya ningún obstáculo tecnológico para ejecutar los actos mencionados anteriormente (formato de datos abiertos). Un formato de Datos abiertos es un formato que pueda ser utilizado sin imponer ninguna restricción (ni de uso monetario, ni de sistema operativo o de otro tipo).
5.. Los datos abiertos pueden exigir como condición de uso para la redistribución y la reutilización el reconocimiento a los contribuyentes y a los creadores de los datos. Este reconocimiento no debe imponerse de manera onerosa.
6.. La licencia puede requerir como condición que la base de datos, si es modificada o procesada para ser distribuida, tenga un nombre que indique que es otra base de datos diferente.
7.La licencia no debe discriminar a ninguna persona o grupo de personas.
8. La licencia no debe restringir a nadie hacer uso de la obra en un ámbito de trabajo específico. Por ejemplo, no puede restringir el uso de la obra en un negocio, o que esta sea utilizada para investigación militar.
9. Los derechos adjuntos a la obra deben aplicarse también a cualquier persona a quién le sea redistribuida sin necesidad de que esta ejecute una licencia adicional

Descarga de datos a través de R y RStudio.

Descarga automatizada 🤖

Descarga de datos a través de R y RStudio.

Ventajas:

- En ciertos casos, te permite hacer las descargas más rápido que si hicieras el trabajo de manera manual
- Es más desafiante intelectualmente 🤓
- Permite a los lectores reproducir tu trabajo
- Permite solo compartir el código, sin tener que subir todos los archivos, dejando la descarga al lector

Funciones a repasar

- **str_c()** , para concatenar texto.

- **curl::curl_download()**, para descargar archivos de internet (principalmente *.zips y *.xlsx)

- **zip::unzip()**, para deszipear o descomprimir archivos de internet.

-Función ***for*** para realizar bucles o ciclos.

curl::curl_download()

Función que sirve para bajar archivos de internet.

curl::curl_download()

Download file to disk

Description

Libcurl implementation of `C_download` (the "internal" download method) with added support for https, ftps, gzip, etc. Default behavior is identical to `download.file`, but request can be fully configured by passing a custom `handle`.

Usage

```
curl_download(url, destfile, quiet = TRUE, mode = "wb", handle =
new_handle())
```

Argumentos importantes

* **url,** la url donde se encuentra el archivo a descargar.

* **destfile,** como se va a llamar el archivo, una vez que lo descargue en la compu.

¿Qué es curl?

`curl` (Client URL) es un programa de línea de comandos **`que transfiere datos desde o hacia un servidor.`**

Navegador web
Visitas una URL → ves la página renderizada (HTML + CSS + imágenes)

`curl`
Visitas una URL → recibes el código fuente crudo (texto, JSON, archivo)

Ejemplo: descargar Información de México:

```
$ curl https://restcountries.com/v3.1/name/mexico

[{"tld":
[".mx"],"cca2":"MX","ccn3":"484","cca3":"MEX","cioc":"MEX","in
dependent":true,"status":"officially-
assigned","unMember":true,"idd":{"root":"+5","suffixes":
["2"]},"capital":["Mexico City”] (…)
```

zip::unzip()

Función que sirve para descomprimir zips.

Ejemplo de uso:

```
zip::unzip("01_Datos/Datos INE/01.zip",
           exdir = "01_Datos/Datos INE")
```

Descompresión de un zip ubicado en mi carpeta 01_Datos/Datos INE llamado “01.zip”

Argumentos importantes:

- ***zipfile**, el nombre y ubicación del archivo *.zip que se va a descomprimir.
- ***exdir**, Directorio en el cual vamos a colocar los archivos que resulten de la descompresión.

curl::curl_download()

Función que sirve para bajar archivos de internet.

Ejemplo de uso:

```
curl::curl_download(url = "https://www.inegi.org.mx/contenidos
/programas/intercensal/2015/tabulados/14_vivienda_mor.xls",
                    destfile =
                      "01_Datos/Datos Censo/HogaresMorelos.xls")
```

Descarga automatizada del tabulado de viviendas de Morelos para la encuesta intercensal 2015.

Función *for*

Función que sirve para crear bucles o ciclos.

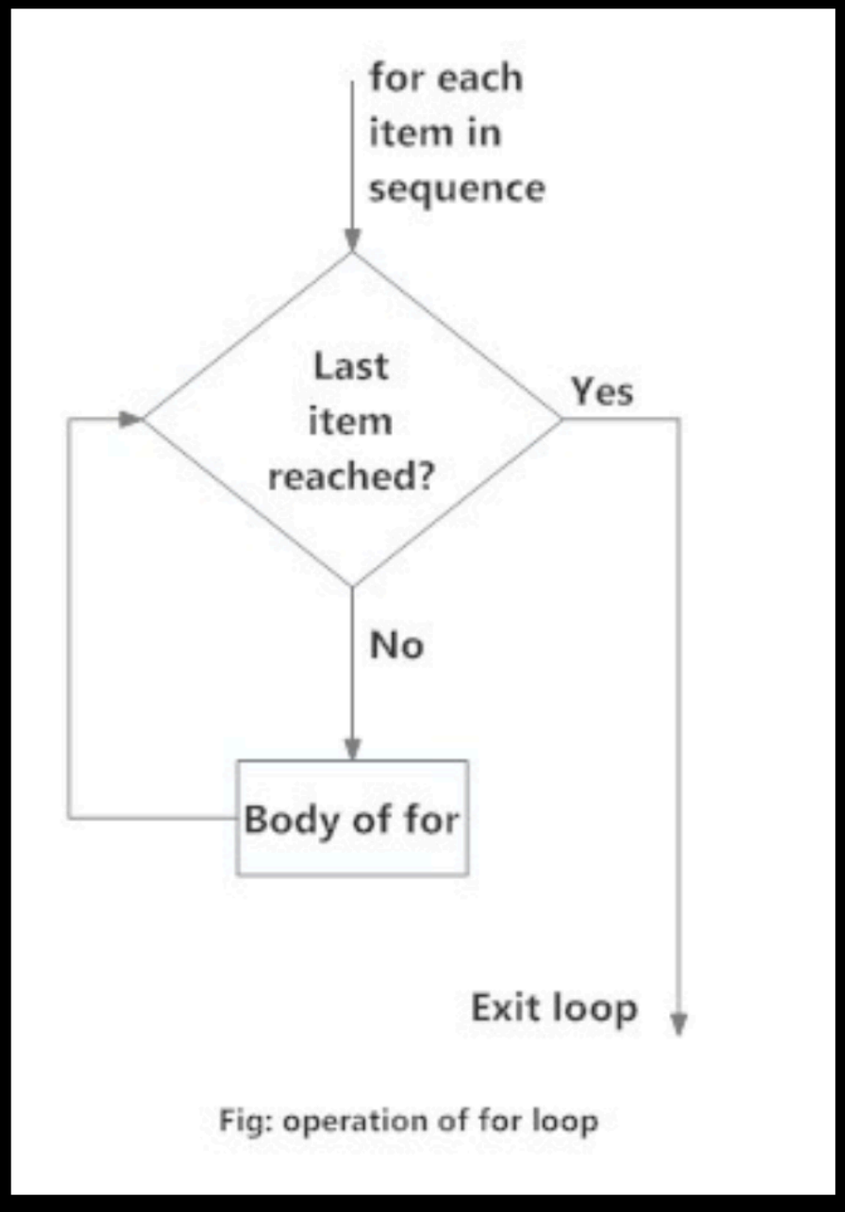


Fig: operation of for loop

Sintaxis

```
for (elemento in secuencia){
    Haz_paso_1
    Haz_paso_2
    ...
    Haz paso_n
}
```

Loops en R con for

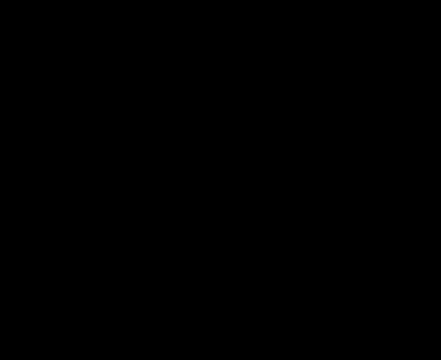
Juvenal Campos.

- Quiero hacer un loop:

1. Defino qué quiero hacer...

... y que resultado esperar

(Generalmente hacemos loops para automatizar tareas repetitivas).



2. Lo hago para un caso $n = 1$.

(Hago ese proceso una vez)

Loops en R con for

Juvenal Campos.

3. Identifico las partes variables del proceso

(Qué cambiaría en el caso $n=2,3,\ldots n$?

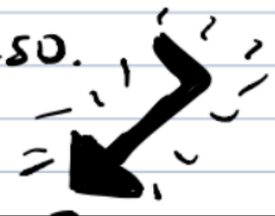
4. Cambio las partes variables por objetos de R.

(para irlo cambiando en el loop)

Loops en R con for

Juvenal Campos.

5. Construyo el for
(o uso algún otro tipo de loop)
definiendo secuencia, contador
y proceso.



⇒ ¿algo tronó?

Sí

(Muchas veces las cosas truenan cuando hacemos un loop. Hay que ver qué excepción surgió cuando lo estábamos corriendo y ajustar en caso de que sea necesario)

No

Que algo no haya tronado no significa que estemos bien. Hay que verificar si el resultado obtenido es el deseado y, si no, modificar el proceso para corregirlo.

Ejercicio práctico 01

Suponga que quiere construir el indicador de "**Grado promedio de educación**" para todos los municipios de México. Para el año 2010 tendría que descargar **todos** los tabulados de INEGI de educación disponibles en este sitio:

https://www.inegi.org.mx/programas/ccpv/2010/#tabulados

Correspondientes a las tablas con la siguiente descripción: "*Población de 15 años y más por municipio, sexo y grupos quinquenales de edad según nivel de escolaridad y grado promedio de escolaridad*"

Establezca un *loop* el cual le permita descargar los datos de manera automatizada.

2. Web Scraping

Concepto: Web Scraping

*"Proceso de **extracción de datos almacenados en la web.** Su objetivo es el de recopilar información almacenada en una página web. Podemos "escrapear" artículos web, e-commerce, obtener precios, reseñas, etc."*

(Platzi).

Concepto: Web Scraping

"La práctica de recolectar datos de manera automatizada a través de internet, *sin recurrir a la interacción con una API o al trabajo de un humano recolectando información".*

(Ryan Mitchell, Web Scraping with Python).

Discusión: ¿Qué tan Legal es el WS?

Si tu respuesta es “Si”, replantéate lo que piensas hacer

1. ¿Estoy violando alguna reglamentación local?
2. ¿Estoy violando los **Términos y Condiciones** del sitio?
3. ¿Estoy accediendo a lugares **no autorizados,** o a lugares donde se necesita hacer **login**?
4. ¿Es legal el uso que le voy a dar a los datos o **genera algún perjuicio de algún modo**?
5. ¿Estoy violando alguna ley de **derechos de autor**?
6. ¿Estoy **accediendo a datos personales**, o a datos que pudieran violar el **secreto comercial**?

El semáforo ético del scraping

Tres niveles de riesgo que se evalúan antes de correr el script

Adelante

Hay datos abiertos o API oficial; robots.txt lo permite;
el sitio es público y se scrapea despacio, identificándose

Con cuidado

HTML público sin API; términos de uso ambiguos;
volumen alto: leer robots.txt, espaciar solicitudes, citar la fuente

Alto

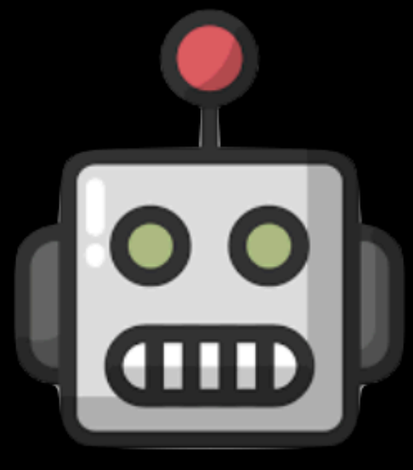
Datos personales sin base legal; contenido tras contraseña
o muro de pago; evadir bloqueos; ignorar un 'no' explícito del sitio

Concepto: Robots.txt

El **robots.txt** es un archivo que define buenas prácticas a la hora de hacer scraping. Nos dice a qué sitios no quiere la página que accedamos.

Es recomendable seguir estas indicaciones, ya que no hacerlo puede ser **no-ético** y puede tener, en algunos casos**, consecuencias legales.**

https://platzi.com/robots.txt
https://www.linkedin.com/robots.txt
https://www.facebook.com/robots.txt
https://www.imdb.com/robots.txt
https://twitter.com/robots.txt



⚠ ¿Qué pasa si ignoras robots.txt?

> **robots.txt no es una barrera técnica** — es una convención social. Puedes ignorarlo, **pero eso no significa que debas hacerlo.**

Tu IP puede ser bloqueada
El servidor detecta tráfico anómalo y bloquea tu dirección. Pierdes acceso al sitio completo.

Consecuencias legales
En algunos países hay precedentes legales contra el scraping no autorizado (ej. caso LinkedIn vs. hiQ Labs en EE.UU.).

Es una cuestión de ética profesional
Como investigadores y analistas de datos, respetar las reglas de acceso es parte de la integridad profesional.

Regla general: si el sitio es gubernamental y los datos son públicos → adelante. Si requiere login, CAPTCHA, o sus términos lo prohíben → busca otra fuente o pide permiso.

Buenas prácticas del *scraping*

1. Revisa robots.txt primero

Antes de *scrapear* cualquier sitio, visita dominio.com/robots.txt y verifica qué secciones están permitidas y cuáles no.

2. Pon pausas entre peticiones

Usa `Sys.sleep(2)` en R entre cada descarga. Bombardear un servidor con peticiones simultáneas puede tumbarlo (o te pueden bloquear).

3. Respeta los términos de servicio

Algunos sitios prohíben el scraping en sus términos legales. Revísalos, sobre todo en redes sociales y sitios comerciales.

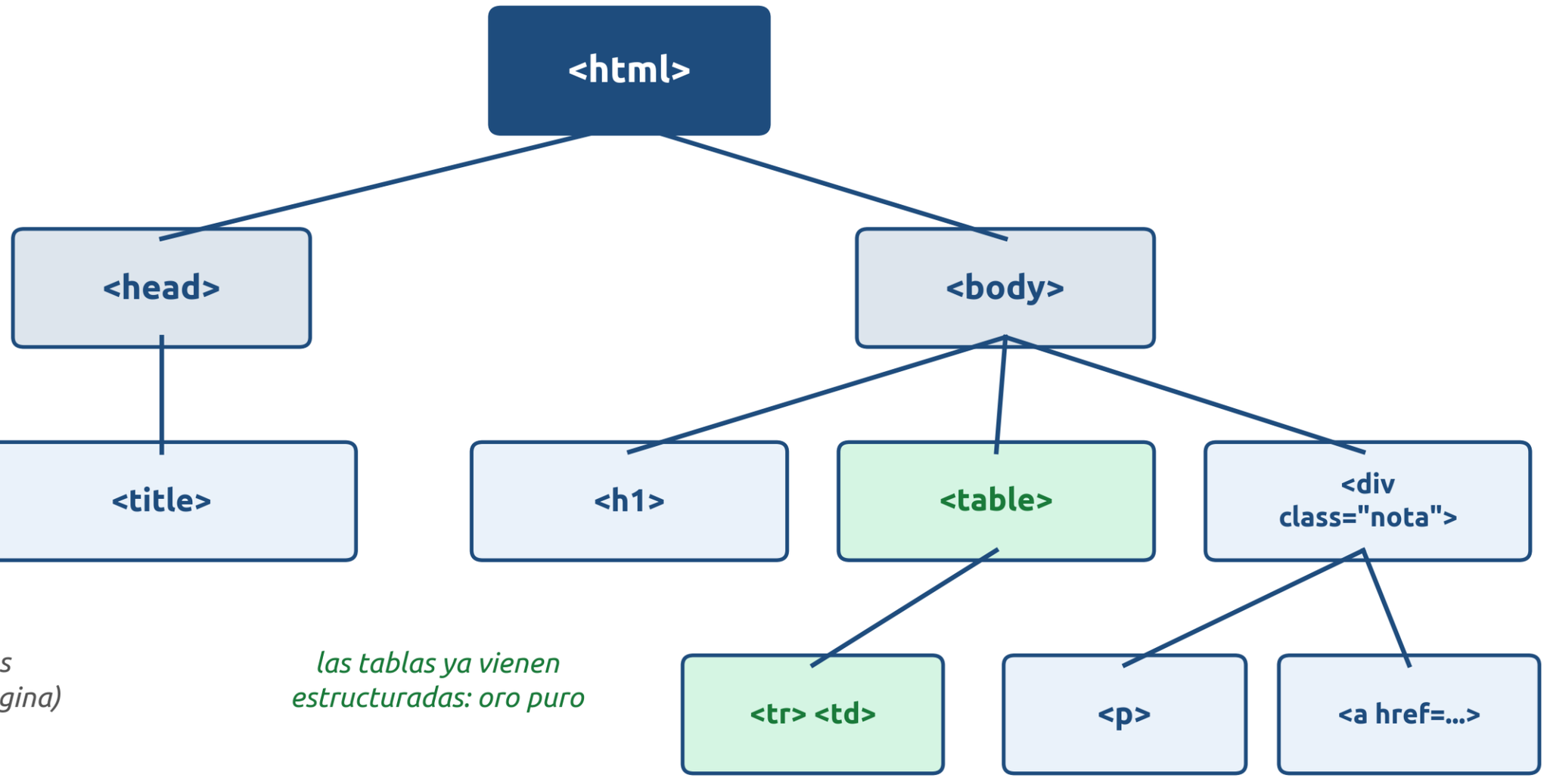
4. Distingue público de privado

Scrapear datos gubernamentales abiertos (INEGI, datos.gob.mx) generalmente es aceptable. Datos tras login o CAPTCHA, no.

Toda página web es un árbol de etiquetas HTML

Una página web es un árbol: el scraping navega sus ramas

La raíz: todo el documento



<head>: metadatos (no se ven en la página)

las tablas ya vienen estructuradas: oro puro

Para verlo en vivo: clic derecho en cualquier página, 'Inspeccionar'.

(Ojo, si usas el navegador *Safari* de Mac, necesitarás activar el modo Desarrollador para habilitar esta opción).

Páginas Web

Árbol

Inspector

```
<!DOCTYPE html>
▼ <html lang="es-mx">
  ▶ <head>…</head>
  ▼ <body>
    ▼ <div class="wrapper">
      ▼ <header class="header">
        ▼ <nav class="nav">
          ▼ <a href="/" class="nav-logo">
              <img src="/images/logo.png" width="50" height="50" alt=
              "Logo">
            </a>
          ▼ <ul class="nav-links">
            ▼ <li>
                <a href="/sobre_mi/">Sobre mí</a>
              </li>
            ▶ <li>…</li>
            ▶ <li>…</li>
            ▶ <li>…</li>
            </ul>
          </nav>
        </header>
      ▼ <section class="hero">
        ▼ <div class="hero-inner">
            <h1>Juve Campos</h1>
            <h2>Blog personal 😎 👨‍💻 </h2>
          </div>
        </section>
      ▼ <main class="content" role="main">
        ▼ <div class="archive">
            <h2 class="archive-title">2020</h2>
          ▼ <article class="archive-item">
              <a href="/2020/07/26/cortando-islas/" class="archive-item-
              link">Cortando Islas</a>
              <span class="archive-item-date"> 2020-07-26 </span>
            </article>
          ▼ <article class="archive-item">
              <a href="/2020/03/03/ejemplos-de-lo-que-se-puede-hacer-en-
              shiny/" class="archive-item-link">Ejemplos de lo que se
              puede hacer en shiny</a>
              <span class="archive-item-date"> 2020-03-03 </span>
            </article>
          ▶ <article class="archive-item">…</article> = $0
          ▶ <article class="archive-item">…</article>
          ▶ <article class="archive-item">…</article>
          ▶ <article class="archive-item">…</article>
          </div>
        </main>
      ▼ <footer class="footer">
        ▶ <ul class="footer-links">…</ul>
        </footer>
      </div>
    </body>
  </html>
```

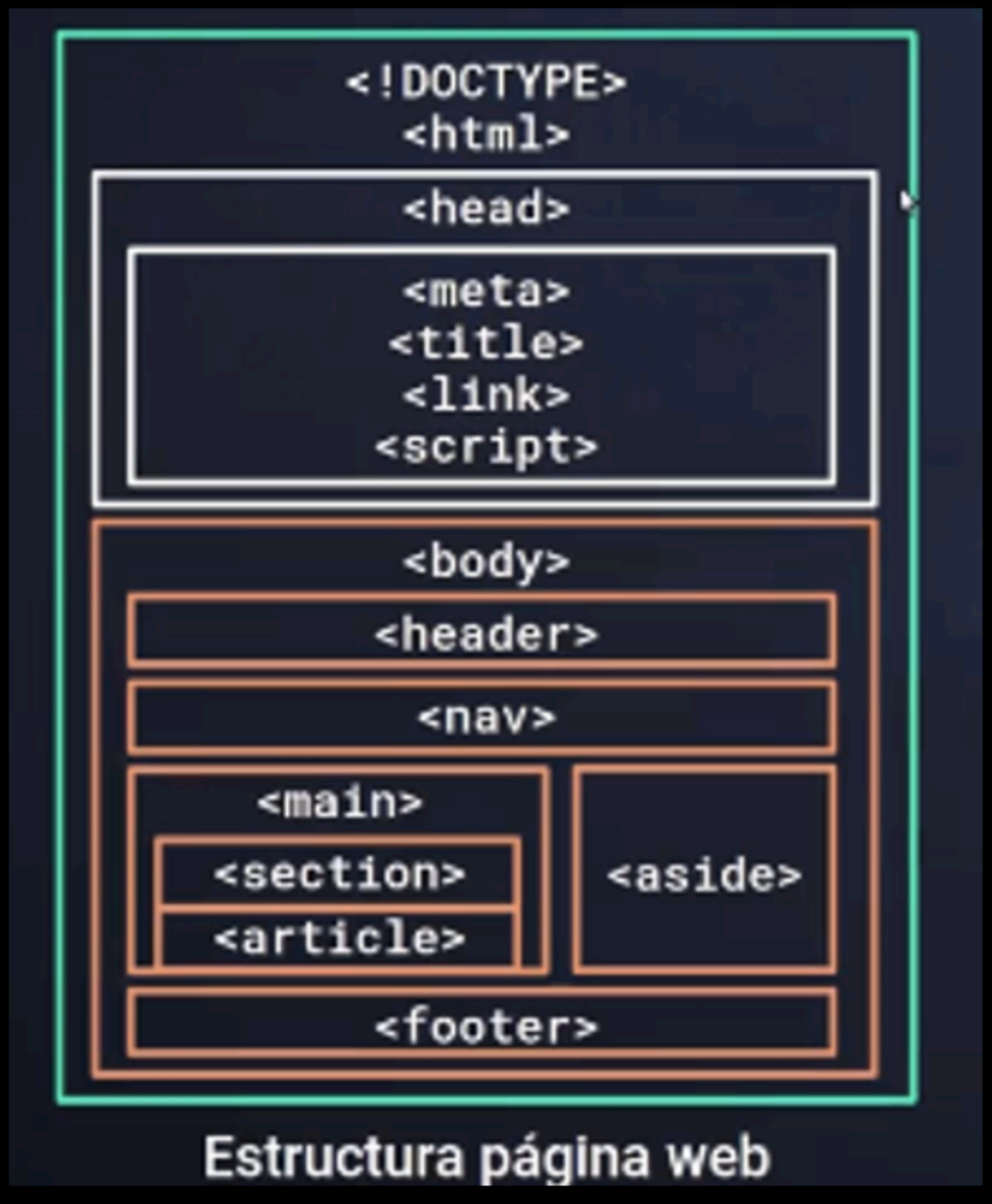
Páginas Web

Inspector

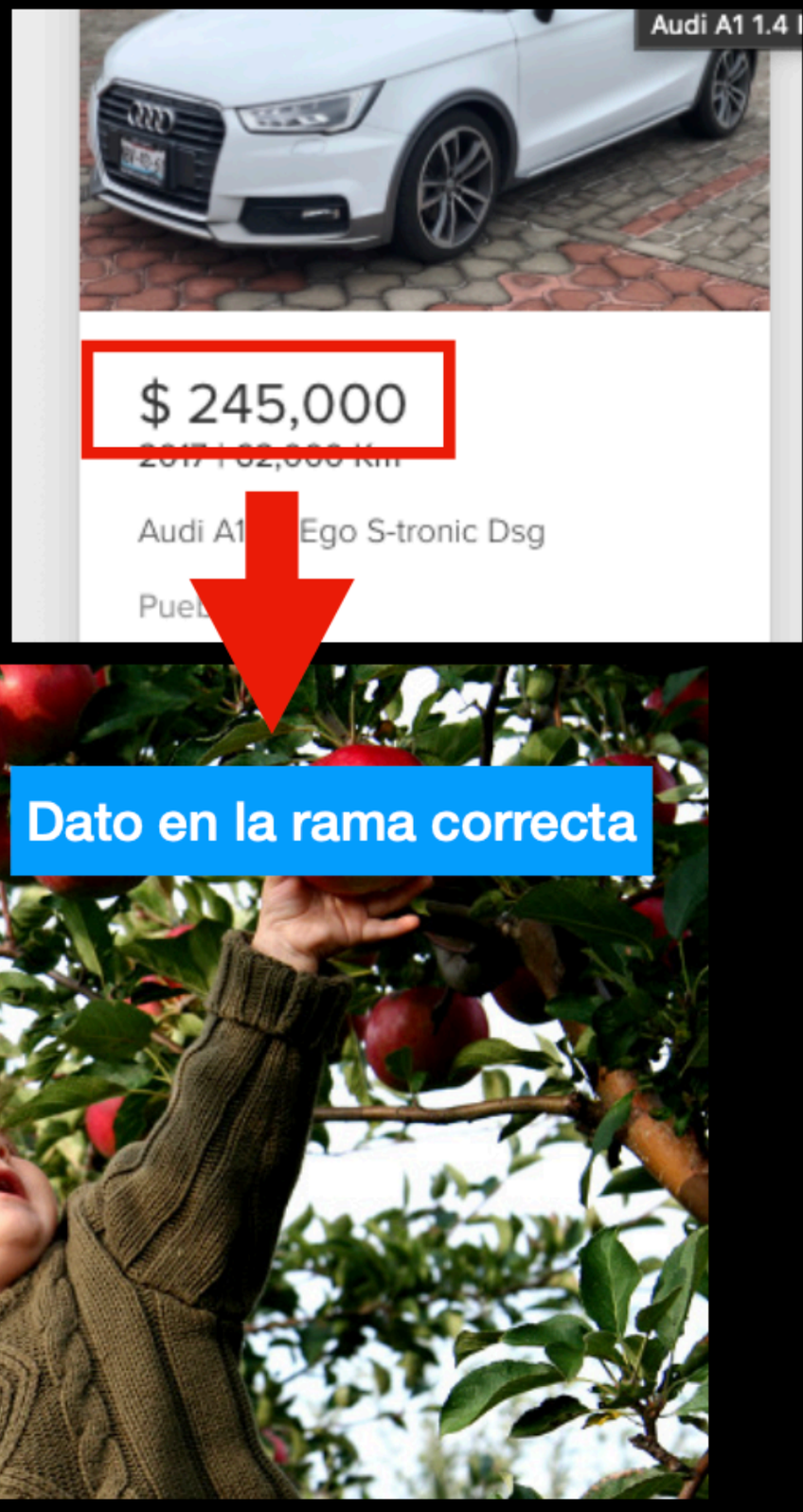
Para entrar al inspector, desde el navegador, hay que apretar click-derecho >> inspeccionar elemento

Si usas el navegador Safari de Mac, esta función viene oculta por defecto.

Páginas Web



Estructura página web



Etiquetas, atributos y clases: los ganchos

Anatomía de un elemento

Leer más

a = etiqueta | class y href = atributos | Leer más = texto

Etiquetas frecuentes

h1...h6: títulos

p: párrafos

a: enlaces (href)

table, tr, td: tablas

div, span: contenedores

img: imágenes (**src =** ***fuente de la imagen***)

Clases e identificadores

class (.) agrupa elementos del mismo tipo visual (muchos por página).

id (#) identifica un elemento único.

Son los dos ganchos favoritos del scraping tradicional.

Páginas Web - Tags

```
<main class="content" role="main">
  <div class="archive">
    <h2 class="archive-title">2020</h2>
    <article class="archive-item">
      <a href="/2020/07/26/cortando-islas/" class="archive-item-
      link">Cortando Islas</a>
      <span class="archive-item-date"> 2020-07-26 </span>
    </article>
    <article class="archive-item">
      <a href="/2020/03/03/ejemplos-de-lo-que-se-puede-hacer-en-
      shiny/" class="archive-item-link">Ejemplos de lo que se
      puede hacer en shiny</a>
      <span class="archive-item-date"> 2020-03-03 </span>
    </article>
    ▶ <article class="archive-item">…</article> = $0
    ▶ <article class="archive-item">…</article>
    ▶ <article class="archive-item">…</article>
    ▶ <article class="archive-item">…</article>
  </div>
</main>
▼ <footer class="footer">
  ▶ <ul class="footer-links">…</ul>
  </footer>
</div>
</body>
</html>
```

Cuando hacemos web-scraping de sitios estáticos, vamos a **navegar en el árbol del HTML**, vamos a **anclarnos de los tags y los atributos,** y vamos a **extraer textos y atributos** como, por ejemplo, enlaces de páginas) **para ponerlos en un objeto de R** (vectores, tibbles o listas)

Texto Atributos del HTML-Tag El HTML-Tag

Páginas Web - Clases

Las clases **son atributos** de los cuales n**os podemos anclar** para hacer referencia a cierto elemento de la página web.

Cuando hagamos referencia a ellos en el web-scraping, **hay que colocarles un punto** al inicio del nombre de la clase, p. Ej, "*.archive-item*" o "*.archive-item-date*"

Selectores CSS

Selectores CSS (lo más práctico):

- Por etiqueta: article, table
- Por clase/ID: .price, #main
- Atributo: a[href*="producto"], img[data-src], meta[property="og:title"]
- Combinadores: .card > h2, .item .price, ul.list > li:nth-child(1)

```
<article class="card" data-id="A123">
  <h2 class="title">Café molido</h2>
  <p class="price">$129.90</p>
  <a class="more" href="/producto/cafe-molido">Ver detalle</a>
  <img loading="lazy" data-src="https://…/cafe.jpg">
  <script type="application/ld+json">{"@type":"Product","name":"Café molido","offers"
</article>
```

Por id: "#contenido"

El elemento único con ese identificador; se antecede con el símbolo de gato.

Por clase: ".archive-item-date"

Todos los elementos con esa clase; el punto inicial es obligatorio. El caso típico: fechas o títulos de una lista de artículos.

Combinados: "div.nota a"

Los enlaces que viven dentro de un div con clase nota: se leen de izquierda a derecha, de padre a hijo.

Por etiqueta: "table"

Todas las tablas de la página. Así se extraen tablas de Wikipedia.

Funciones de R

Funciones de R

La librería rvest **nos ayuda a hacer web-scraping.**

Esta librería está **diseñada para trabajar con el tidyverse (%>%)** .

Trata de hacer **fácil las tareas más comunes de web-scraping**, inspirada en la librería *beautifulsoup* de Python.

El flujo rvest: leer una vez, seleccionar, extraer

Cinco pasos: del HTML crudo al tibble ordenado

con cortesía: Sys.sleep(2) entre páginas

aquí entra el selector: "table", ".nota", "#titulo"

de aquí en adelante: filter(), mutate(), count()

read_html() se llama UNA vez y el resultado se guarda en un objeto.

Se guarda el código de la página web en un objeto de R que se puede consultar múltiples veces.

Probar selectores sobre ese objeto no vuelve a molestar al sitio.

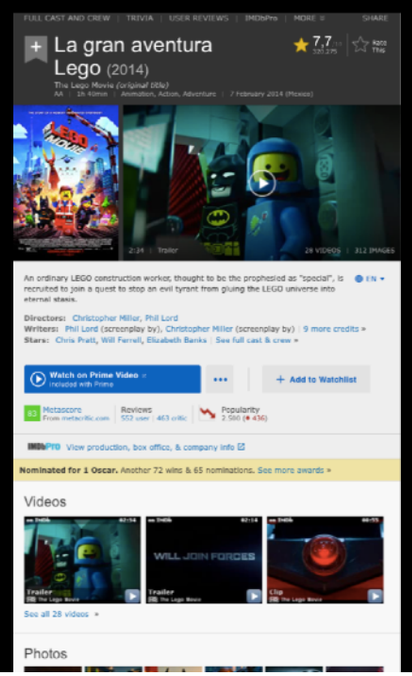
rvest::read_html(url)

Esta función nos permite descargar a nuestra sesión de R el código HTML de la página a la que le queremos sacar información.

```
library(rvest)

lego_movie <- read_html("http://www.imdb.com/title/tt1490017/")
```

Argumento: **url, cadena de texto.**
La dirección web de la página a la cual le queremos extraer información.



rvest::html_nodes()

Esta función extrae un nodo (rama) del código de la página web que tenemos almacenada en un objeto generado por *rvest::read_html()*

```
rating <- lego_movie %>%
  html_nodes("strong span")
```

**Atrapa los nodos del tipo "strong span" dentro del código html.*

El resultado de esta función aún no es legible. Hay que transformarlo (como lo que nos salía de las APIs).

rvest::html_nodes()

Los nodos pueden ser uno de los siguientes datos:

A) Un selector CSS.
B) Un tag de HTML
C) Un XPath (Dirección a un elemento en particular)
D) Una ruta de selección

Y se pueden combinar entre ellos.

rvest::html_text()

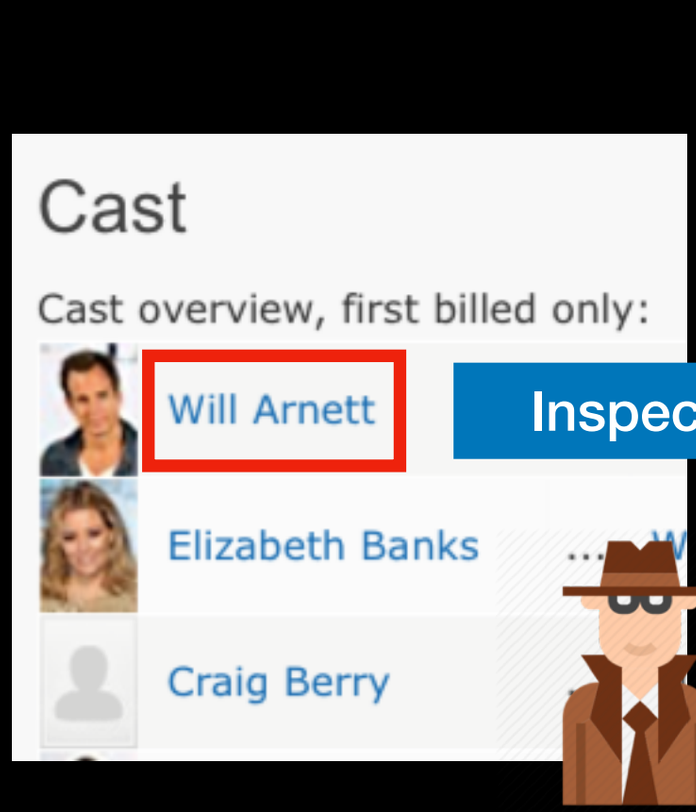
Esta función extrae el contenido que se encuentra dentro de un nodo (rama), resultante de la función *rvest::html_node()* o *rvest::html_nodes()*.

```
rating <- lego_movie %>%
  html_nodes("strong span") %>%
  html_text() %>%
  as.numeric()
rating
#> [1] 7.7
```

Acá la rama seleccionada es un tag de HTML

rvest::html_attr()

Esta función extrae el atributo HTML de un nodo que almacene atributos.



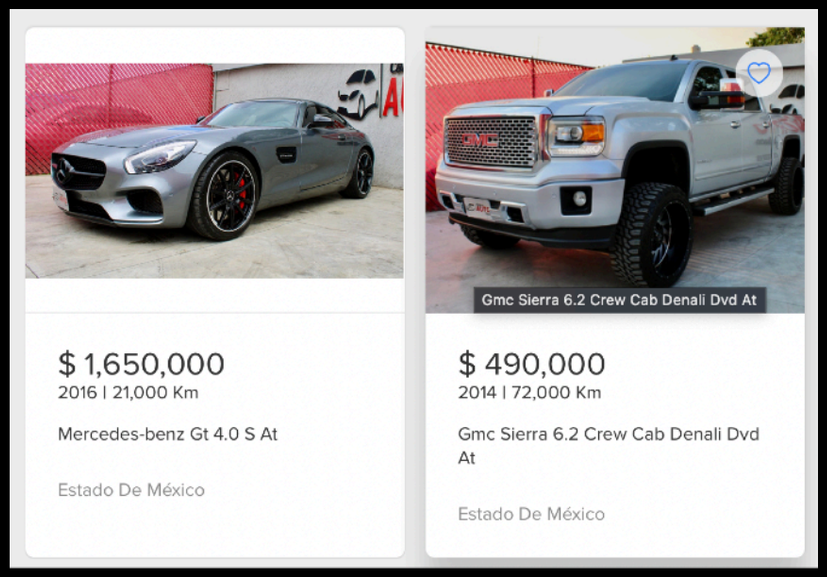
rvest::html_attr()

Ejemplos

```
cast <- lego_movie %>%
  html_nodes("#titleCast .primary_photo img") %>%
  html_attr("alt")
cast
#>  [1] "Will Arnett"     "Elizabeth Banks" "Craig Berry"     "Alison Brie"
#>  [5] "David Burrows"   "Anthony Daniels" "Charlie Day"     "Amanda Farinos"
#>  [9] "Keith Ferguson"  "Will Ferrell"    "Will Forte"      "Dave Franco"
#> [13] "Morgan Freeman"  "Todd Hansen"     "Jonah Hill"
```

```
poster <- lego_movie %>%
  html_nodes(".poster img") %>%
  html_attr("src")
poster
#> [1] "https://m.media-amazon.com/images/M/MV5BMTg4MDk1ODExN15BMl5BanBnXkFtZTgwNzIyNjg3MDE@._V1_U
```

rvest::html_text()



```
▼ <a href="https://auto.mercadolibre.com.mx/MLM-903077082-mercedes-benz-clase-
gt-s-am2016-factura-original-tomo-auto-_JM#position=1&type=item&
tracking_id=04316880-e372-4fe8-92e6-faa5eabc400b" class="ui-search-
result__content ui-search-link" title="Mercedes-benz Gt 4.0 S At">
  ▼ <div class="ui-search-result__content-wrapper">
    ▼ <div class="ui-search-item__group ui-search-item__group--price">
      ▼ <div class="ui-search-price ui-search-price--size-medium ui-search-
      item__group__element">
        ▼ <div class="ui-search-price__second-line">
          ▼ <span class="price-tag ui-search-price__part">
              <span class="price-tag-symbol">$</span>
              <span class="price-tag-fraction">1,650,000</span>
            </span>
          </div>
        </div>
      </div>
    ▼ <div class="ui-search-item__group ui-search-item__group--attributes">
      ▼ <ul class="ui-search-card-attributes ui-search-item__group__element">
        ▶ <li class="ui-search-card-attributes__attribute">…</li>
          <li class="ui-search-card-attributes__attribute">21,000 Km</li>
        </ul>
      </div>
    ▼ <div class="ui-search-item__group ui-search-item__group--title">
        <h2 class="ui-search-item__title ui-search-
        item__group__element">Mercedes-benz Gt 4.0 S At</h2>
      </div>
    ▼ <div class="ui-search-item__group ui-search-item__group--location">
        <span class="ui-search-item__group__element ui-search-
        item__location">Estado De México</span>
      </div>
    </div>
</a>
```

Si ya encontramos el dato que estamos buscando, y ya localizamos la rama correcta, lo podemos extraer con la función html_text()

rvest::html_text()

```
precios = read_html(url) %>%
  html_nodes(".andes-card") %>%
  html_nodes(".ui-search-result__content-wrapper") %>%
  html_nodes(".ui-search-price") %>%
  html_text() %>%
  str_remove_all(pattern = "\\$|\\,") %>%
  as.numeric()
```

```
carro = tibble(carro = read_html(url) %>%
                 html_nodes(".andes-card") %>%
                 html_nodes(".ui-search-result__content-wrapper") %>%
                  html_nodes("h2") %>%
                  html_text()) %>%
  mutate(marca = str_extract(carro, pattern = "\\w+"))
```

Si ya encontramos el dato que estamos buscando, y ya localizamos la rama correcta, lo podemos extraer con la función html_text()

rvest::html_table()

Country/Territory	Recreational	Medical	Notes
Afghanistan	Illegal	Illegal	*Main article: Cannabis in Afghanistan* Production banned by King Zahir Shah in 1973.[9]
Albania	Illegal	Illegal	*Main article: Cannabis in Albania* Prohibited but plants highly available throughout the country and law often unenforced.[10][11][12]
Algeria	Illegal	Illegal	*Main article: Cannabis in Algeria*
Andorra	Illegal	Illegal	*Main article: Cannabis in Andorra*
Angola	Illegal	Illegal	*Main article: Cannabis in Angola*
Antigua and Barbuda	Decriminalized	Illegal	*Main article: Cannabis in Antigua and Barbuda*
Argentina	Decriminalized	Legal	*Main article: Cannabis in Argentina* Decriminalized for small amounts and private consumption, as ruled by the Supreme Court in 2009.[13] Medicinal cannabis legal nationally since 21 September 2017.[14]
Armenia	Illegal	Illegal	*Main article: Cannabis in Armenia*
Australia	Decriminalized in Northern Territory and South Australia.[15][16] Legal in Australian Capital Territory for personal use but not for sale.	Legal at federal level and in all states.[17] Qualifying conditions and other details vary by state.[18]	*Main article: Cannabis in Australia* In September 2019, the Australian Capital Territory became the first state or territory of Australia to legalize recreational use of cannabis. Since 31 January 2020 residents have been allowed to grow two plants and possess 50 g, though sales or other transfer is prohibited, including cannabis seeds. Federal law also remains enforceable.[19]

Url = https://en.wikipedia.org/wiki/Legality_of_cannabis

Esta función nos sirve si en la página web ya esta ordenada la función que buscamos en forma de tabla. Nos extrae todas las tablas del sitio y nos las acomoda en una lista.

HTML tags más comunes

- **<h1>, h2, h3, h4, h5, h6:** Sirven para añadir y especificar títulos (y subtítulos) de la página.

- **<p>:** Sirve para agregar un párrafo.

- **:** Pone el texto en negrita.

- **<img>:** Añade imágenes.

- **<body>:** Para agregar contenido a la página.

- ** :** Ayuda con la división dentro del contenido.

- **<a href = “”>:** Sirve para agregar enlaces

- **
:** Sirve para agregar una sola línea de texto.

- **<b>:** Texto en negritas.

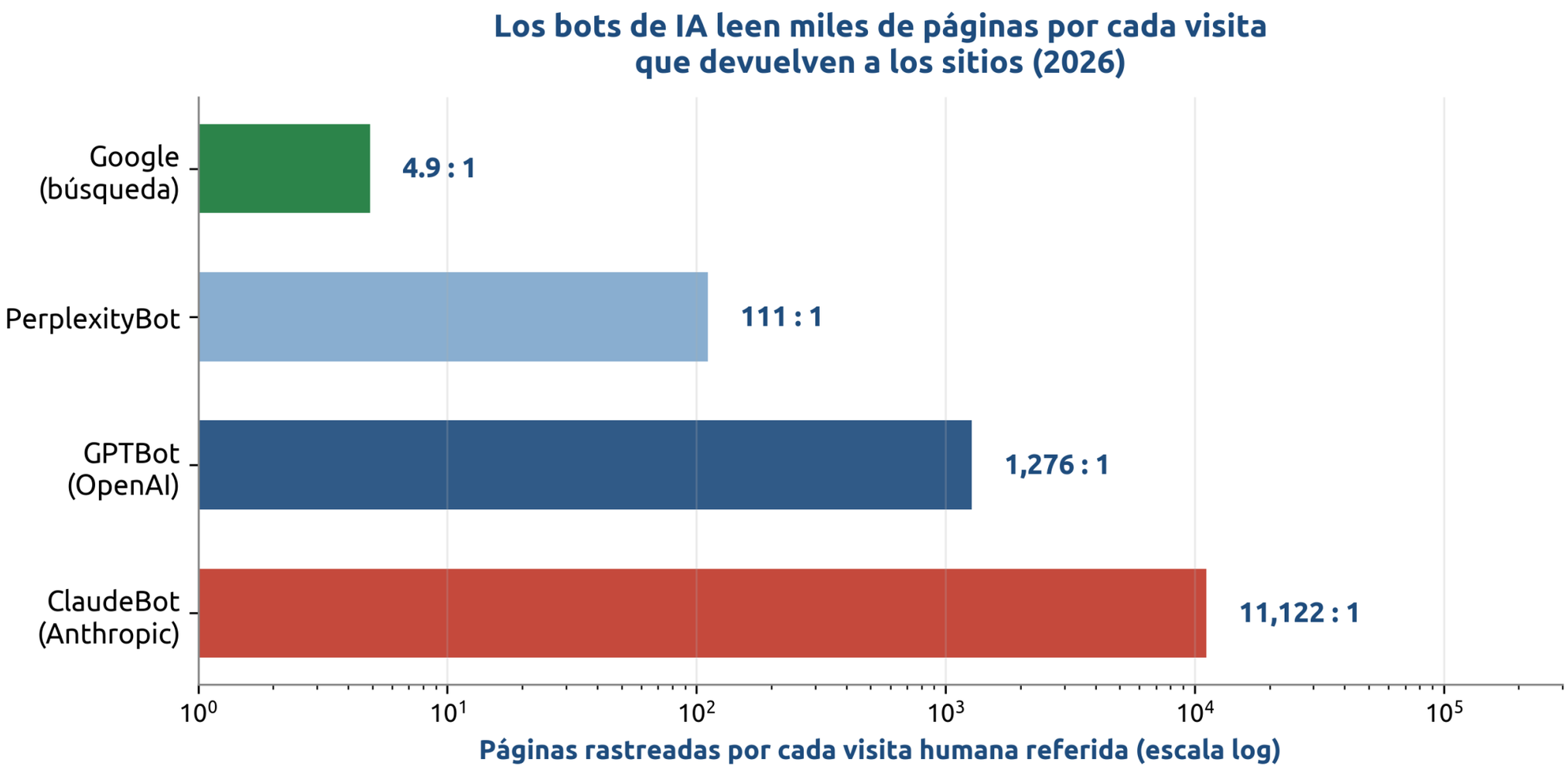
- **<ul>:** Listas no ordenadas o bullets.

- **<ol>:** Listas ordenadas o numeradas.

Ejercicio práctico 02

1. Entre al sitio “https://en.wikipedia.org/wiki/Legality_of_cannabis" y extraiga la tabla de países donde el consumo de cannabis es ilegal. **¿En qué porcentaje de países el consumo recreacional es ilegal?**
2. Entre al sitio “https://atiempo.tv/author/juvenal-campos/" y extraiga **todos** los artículos de este autor.
3. Entre al sitio de casas de ***MercadoLibre*** y descargue los precios y la información de las casas que hay disponibles en el sitio.
4. Entre al sitio de simulación de créditos de la Condusef (https://phpapps.condusef.gob.mx/condusefhipotecario/datos.php) y descargue la mayor cantidad de simulaciones posibles para préstamos de una casa.

Los bots ya generan más tráfico que los humanos



En junio de 2026 los bots generaron 57.5% del tráfico HTML (los humanos, 42.5%). El trato implícito de la web (te rastreo y te mando visitas) se rompió: los bots de IA leen miles de páginas y regresan casi nada.

Fuente: compilación de Digital Applied sobre Cloudflare Radar (consultada el 1 de agosto de 2026).

La web responde: bloqueo por defecto y peajes

Bloqueo por defecto

Desde julio de 2025 Cloudflare bloquea a los rastreadores de IA salvo permiso explícito: el modelo pasó de opt-out a opt-in, con el respaldo de más de 50 medios y plataformas.

Pago por rastreo (pay per crawl)

El sitio cobra por solicitud usando el código HTTP 402 (Payment Required); el bot declara cuánto está dispuesto a pagar. En beta desde 2025.

Qué significa para ustedes

El scraping casual de sitios grandes será cada vez más difícil: más razón para preferir datos abiertos y APIs, y para scrapear solo donde son bienvenidos.

Fuente: comunicados y blog técnico de Cloudflare, julio de 2025 (consultados el 1 de agosto de 2026).

Tecnológico de Monterrey

httr2 y precauciones

Qué hacer cuando read_html() ya no alcanza

Ciencia de Datos II. Complemento de la Clase 16
Jorge Juvenal Campos Ferreira

Extraer datos reales exige más que read_html()

El raspado web (web scraping) extrae datos de páginas con código.

Situación

read_html() descargó sin problema los 133 artículos de Atiempo.

Complicación

El mismo método falló en gob.mx con un mensaje que no explicaba la causa.

Resolución

httr2 controla la petición, y seis precauciones protegen al sitio y a los datos.

Pregunta de la sesión

¿Qué cambia cuando el servidor deja de comportarse como esperamos?

En Atiempo, read_html() alcanzó para todo

El sitio entrega HTML normal y acepta la petición que envía R.

12

páginas recorridas

133

artículos extraídos

0

errores de conexión

```
html_juve <- read_html(url_juve)

titulos <- html_juve %>%
  html_nodes(".entry-title") %>%
  html_nodes("a") %>%
  html_text()
```

Cuando todo sale bien, una sola función oculta toda la conversación con el servidor.

Fuente: atiempo.tv, extracción propia del 4 de noviembre de 2026.

En gob.mx, la misma función falla sin explicar

El mensaje no dice si falló la URL, internet o el servidor.

```
pagina <- paste0(
  "https://www.gob.mx/presidencia/es/archivo/articulos",
  "?idiom=es&order=DESC&page=2"
)
codigo_pagina <- read_html(pagina)
```

```
Error in open.connection(x, "rb") :
  cannot open the connection
```

Lo que vimos en RStudio

La conexión no abrió, y R no mostró ningún código del servidor.

La pista que faltaba

Desde curl, la misma URL respondió con el código 200 (éxito).

El servidor rechaza al identificador de RStudio

El identificador (User-Agent) es el texto con el que el cliente se presenta.

Identificador enviado	Resultado
R (4.6.1 ...)	**Funciona**
Mozilla/5.0	**Funciona**
RStudio	**Funciona**
RStudio Desktop	**403**
RStudio Desktop (2026.7.1.147); R (...)	**403**

Qué significa

El código 403 indica que el servidor entendió la petición y se negó a atenderla. La regla exacta sigue sin conocerse.

Fuente: pruebas propias contra gob.mx, 4 de noviembre de 2026, con identificadores simulados.

Cada petición HTTP lleva datos que el servidor lee

HTTP es el protocolo de la web; req_dry_run() muestra la petición sin enviarla.

```
request(pagina) %>%
  req_user_agent("Mozilla/5.0") %>%
  req_dry_run()
```

```
GET /presidencia/es/archivo/articulos HTTP/1.1
accept: */*
accept-encoding: deflate, gzip
host: www.gob.mx
user-agent: Mozilla/5.0
```

GET

Este método pide un recurso al servidor.

host

Este campo nombra el sitio que recibe la petición.

user-agent

Este campo dice quién pregunta. Aquí estaba el problema.

Fuente: salida de httr2 1.3.0.

httr2 separa la petición en cuatro pasos

read_html() hace todo junto; httr2 te deja intervenir en cada paso.

request(url)	Prepara la petición con la URL. Todavía no envía nada.
req_user_agent(...)	Define el identificador con el que R se presenta.
req_perform()	Envía la petición y recibe estado, encabezados y cuerpo.
resp_body_string()	Saca el cuerpo de la respuesta como una cadena de texto.

Con httr2, el error ya dice qué pasó

El mismo rechazo se lee distinto en cada paquete.

read_html()

cannot open the connection

req_perform()

HTTP 403 Forbidden.

```
respuesta <- request(pagina) %>%
  req_user_agent(identificador_rstudio) %>%
  req_error(is_error = function(resp) FALSE) %>%
  req_perform()

resp_status(respuesta)   # 403
resp_status_desc(respuesta)   # "Forbidden"
```

Un diagnóstico preciso evita buscar la falla en el lugar equivocado.

Fuente: pruebas propias con httr2 1.3.0 y rvest; 403 definido en RFC 9110.

Una respuesta 200 no garantiza que sea HTML

Con el identificador resuelto apareció un segundo problema.

```
<httr2_response>
Status: 200 OK
Content-Type: text/javascript
Body: In memory (10863 bytes)
```

```
$('#prensa').append('<div class="row clearfix
bottom-buffer">  <div class=\"col-md-4\">\n
<article>\n ...
```

Qué pasa con read_html()

No marca error, pero los enlaces salen como \"/presidencia/...\" y los títulos traen basura.

Qué conviene revisar

Content-Type dice qué trae el cuerpo, y resp_content_type() lo consulta en httr2.

Fuente: gob.mx, extracción propia del 4 de noviembre de 2026.

Tres reemplazos recuperan el HTML escondido

resp_body_string() entrega el texto para limpiarlo antes de read_html().

En la respuesta	Se convierte en
\"	"
\/	/
\n	un espacio

```
html_pagina <- texto_pagina %>%
  str_replace_all('\\\\"', '"') %>%
  str_replace_all("\\\\/", "/") %>%
  str_replace_all("\\\\n", " ") %>%
  read_html()
```

Resultado

Cada página del listado entregó 9 artículos con título, fecha y enlace limpios.

Fuente: gob.mx, extracción propia del 4 de noviembre de 2026.

Las pausas protegen al sitio y a tu acceso

Un bucle sin pausas dispara peticiones tan rápido como puede.

125

peticiones en total:
25 de listado y 100 de texto

125 s

de espera como mínimo,
con 1 segundo de pausa

```
for (i in 1:nrow(tabla_conferencias)) {
  # ... descargar y extraer el texto ...
  Sys.sleep(1)
}
```

Por qué importa

El servidor atiende también a otras personas. Además, un ritmo agresivo puede provocar que el sitio bloquee tu dirección.

Fuente: scraping_sheinbaum.R.

Filtrar exige revisar lo que se descarta

El listado mezcla conferencias con giras y otros eventos.

225

artículos revisados en 25 páginas

103

títulos con "conferencia de prensa"

100

conferencias más recientes conservadas

Lo que falló en el primer intento

El patrón largo dejó fuera este título: "Conferencia de prensa matutina del 21 de agosto de 2026".

```
filter(str_detect(
  titulo,
  regex("conferencia de prensa", ignore_case = TRUE)
))
```

Fuente: gob.mx, páginas 1 a 25 del archivo, extracción propia del 4 de noviembre de 2026.

Validar evita confiar en datos incompletos

Un script que termina sin error todavía puede estar mal.

```
stopifnot(
  nrow(tabla_final) == numero_conferencias,
  !any(is.na(tabla_final$texto))
)
```

100 filas

El script reunió las conferencias pedidas.

0 textos vacíos

Todas las conferencias traen su versión.

0 fechas repetidas

Ninguna conferencia aparece dos veces.

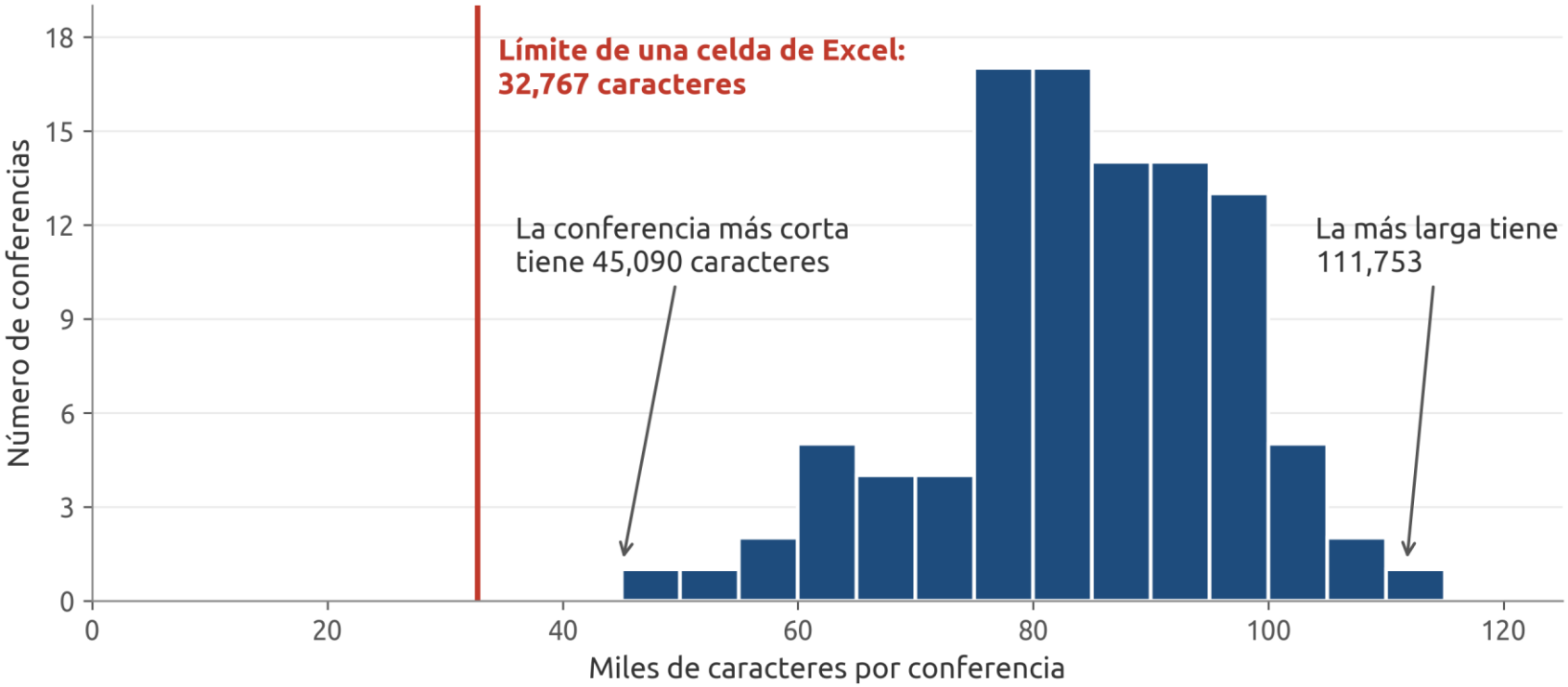
Falta confirmar con otra fuente

Cuatro días hábiles no aparecen: 1, 15 y 16 de septiembre y 2 de noviembre de 2026.

Fuente: gob.mx, extracción propia del 4 de noviembre de 2026.

Las 100 conferencias rebasan el límite de Excel

Por eso el script guarda en CSV y se lee con read_csv().



Atiempo sí cabe en xlsx: su texto más largo ronda 16 mil caracteres.

Fuente: gob.mx, cálculos propios. Límite: Microsoft (s. f.).

Cambiar el identificador también tiene costo ético

La información es pública, pero el filtro lo puso el sitio.

Lo que hicimos

Enviamos otro identificador y con eso esquivamos un filtro del sitio.

Lo que no sabemos

Desconocemos si el sitio quiso bloquear a RStudio o si la regla lo alcanza por accidente.

Lo que conviene revisar antes

Revisa robots.txt: el de gob.mx solo menciona rastreadores de Google.
Lee los términos de uso del sitio.
Busca primero una API o datos abiertos oficiales.
Detente si el sitio pide inicio de sesión o CAPTCHA (prueba contra robots).

Fuente: gob.mx/robots.txt; Koster et al. (2022).

MercadoLibre obliga a buscar otra vía

La misma pregunta técnica recibe otra respuesta en un sitio comercial.

Criterio	gob.mx	MercadoLibre
robots.txt	Solo menciona rastreadores de Google	Veta a todos _Desde_ y _NoIndex_True
Sitio	Gobierno con información pública	Comercio con anuncios de terceros
Raspado	Sin regla general en robots.txt	Declara medidas para impedirlo
API	No se revisó	Exige aplicación registrada y token
Decisión	**Raspar con pausas**	**No raspar: usar API o pedir permiso**

Una API con token no garantiza los datos

Fuentes de terceros reportan que la búsqueda general sigue restringida aun con token. Falta confirmarlo con una aplicación registrada.

Fuente: robots.txt de ambos sitios, 4 de noviembre de 2026; Mercado Libre (s. f.-a, s. f.-b).

httr2 ya incluye pausas y reintentos

Estas funciones reemplazan el Sys.sleep() manual.

```
respuesta <- request(url_base) %>%
  req_url_query(idiom = "es", order = "DESC", page = k) %>%
  req_user_agent("Mozilla/5.0") %>%
  req_throttle(capacity = 1, fill_time_s = 1) %>%
  req_retry(max_tries = 3) %>%
  req_perform()
```

req_url_query()

Arma los parámetros de la URL sin pegar cadenas a mano.

req_throttle()

Limita el ritmo. Así, 25 páginas tardaron 24.5 segundos.

req_retry()

Repite la petición hasta tres veces si ocurre una falla pasajera.

Fuente: prueba propia con httr2 1.3.0, 4 de noviembre de 2026.

Cada precaución evita un riesgo concreto

Las seis precauciones del script responden a seis riesgos.

Precaución	Riesgo que evita
Fijar el identificador	Recibir un rechazo 403
Revisar Content-Type	Extraer basura sin ver un error
Pausar entre peticiones	Saturar el sitio o perder acceso
Auditar el filtro	Perder registros en silencio
Validar con stopifnot()	Confiar en datos incompletos
Guardar en CSV	Cortar textos largos en Excel

Estas fuentes respaldan lo visto en la sesión

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). RFC Editor. https://www.rfc-editor.org/rfc/rfc9110

Koster, M., Illyes, G., Zeller, H., & Sassman, L. (2022). *Robots Exclusion Protocol* (RFC 9309). RFC Editor. https://www.rfc-editor.org/rfc/rfc9309

Mercado Libre. (s. f.-a). *Autenticación y autorización*. Mercado Libre Developers. https://developers.mercadolibre.com.ar/es_ar/autenticacion-y-autorizacion

Mercado Libre. (s. f.-b). *¿Cómo pueden usar otros usuarios tus datos?* https://www.mercadolibre.com.ar/privacidad/declaracion-privacidad/31256

Microsoft. (s. f.). *Excel specifications and limits*. Microsoft Support. https://support.microsoft.com/en-us/office/excel-specifications-and-limits-1672b34d-7043-467e-8e27-269d656771c3

Wickham, H. (2025). *rvest: Easily harvest (scrape) web pages* (versión 1.0.5) [Software]. https://CRAN.R-project.org/package=rvest

Wickham, H. (2026). *httr2: Perform HTTP requests and process the responses* (versión 1.3.0) [Software]. https://CRAN.R-project.org/package=httr2

Conclusiones: el raspado es también diagnóstico

1. read_html() basta cuando el sitio entrega HTML y acepta tu petición.
2. httr2 separa la petición y muestra por qué falló.
3. Un código 200 no garantiza que recibiste el contenido esperado.
4. Las pausas y las revisiones de términos protegen al sitio y a tu trabajo.
5. Validar los datos y auditar los filtros detecta errores silenciosos.